KUBERNETES 1.30 集群部署文档（UBUNTU 22.04，大陆镜像源，无环境变量）

本文档命令在 **Ubuntu 22.04 LTS x86_64** 下执行；IP 与路径均 **写死为下列规划**，不再使用 export / 环境变量。请用 **sudo -i 进入 root**，或在每条命令前加 **sudo**。

0. 大陆镜像与网络说明

用途	镜像 / 做法
Kubernetes 的 deb 仓库（kubelet/kubeadm/kubectl 1.30）	**清华大学 TUNA：** mirrors.tuna.tsinghua.edu.cn/kubernetes/core:/stable:/v1.30/deb/
kubeadm init --kubernetes-version	使用本机已安装 **kubeadm version -o short** 的输出，**无需访问** dl.k8s.io
Calico YAML（raw.githubusercontent.com 加速）	使用 **ghproxy** 前缀（若失效可换其他 GitHub 加速或本地下载后 kubectl apply -f）
Helm 安装包	**华为云镜像：** mirrors.huaweicloud.com/helm/...
Ubuntu 系统包（可选）	将 sources.list 中官方源改为 **阿里云 / 清华 / 中科大** 等（见 2.0）
Helm Chart 仓库（*.github.io）	部分网络需 **代理**；无法访问时见 **7.0 离线/代理**
kubeadm 拉取的容器镜像（registry.k8s.io，实际走 Google Artifact Registry）	大陆直连常 **超时**；Master 使用 **kubeadm init --image-repository=registry.aliyuncs.com/google_containers**；Worker 使用 **join.yaml 中的 ClusterConfiguration.imageRepository**（见 **§4**），与 Master **同一仓库**

以下 IP 为固定规划：

- Master：**192.168.10.166**
- Worker：**192.168.11.116**
- Worker + NFS：**192.168.11.160**
- NFS 导出路径：**/export/k8s-data**
- Pod CIDR：**10.244.0.0/16**

1. 架构与网段

角色	IP
Master	192.168.10.166
Worker	192.168.11.116
Worker + NFS	192.168.11.160

- Pod CIDR：10.244.0.0/16
- Service CIDR：kubeadm 默认 10.96.0.0/12

2. 所有节点通用准备（三台均执行）

2.0（可选）Ubuntu apt 改用阿里云镜像

在每台机器上（**Jammy 22.04** 示例，若你使用其他源请略过）：

```
sudo cp -a /etc/apt/sources.list /etc/apt/sources.list.bak
sudo sed -i 's@http://archive.ubuntu.com/ubuntu@https://mirrors.aliyun.com/ubuntu@g' /etc/apt/sources.list
sudo sed -i 's@http://security.ubuntu.com/ubuntu@https://mirrors.aliyun.com/ubuntu@g' /etc/apt/sources.list
```

也可改用清华、中科大等镜像，以本单位规范为准。

2.1 /etc/hosts（可选）

```
sudo tee -a /etc/hosts <<'EOF'
192.168.10.166 k8s-master
192.168.11.116 k8s-worker-1
192.168.11.160 k8s-worker-2
EOF
```

2.2 关闭 swap

```
sudo swapoff -a
sudo sed -i '/ swap / s/^/#/' /etc/fstab
```

2.3 内核模块与 sysctl

```
sudo tee /etc/modules-load.d/k8s.conf <<EOF
overlay
br_netfilter
EOF
sudo modprobe overlay
sudo modprobe br_netfilter

sudo tee /etc/sysctl.d/k8s.conf <<EOF
net.bridge.bridge-nf-call-iptables  = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward                 = 1
EOF
sudo sysctl --system
```

2.4 安装 containerd 并启用 systemd cgroup

版本建议： Kubernetes **1.30** 与官方文档长期针对 **containerd 1.7.x** 验证。若 containerd --version 显示 **v2.x**（如 2.2.x），crictl / kubeadm 可能出现 **unknown service runtime.v1.RuntimeService / ImageService**（CRI 未按预期暴露）。**推荐**使用发行版提供的 **containerd 1.7** 包；若已误装 2.x，见 **§2.4.2** 降级。

```
sudo apt-get update
sudo apt-get install -y containerd

containerd --version
```

若需 **固定为 1.7 系列**（版本号以 apt-cache 为准）：

```
apt-cache madison containerd | head -20
# 选择带 1.7 的版本号，例如：
# sudo apt-get install -y containerd=1.7.x-0ubuntu...

sudo mkdir -p /etc/containerd
sudo sh -c 'containerd config default > /etc/containerd/config.toml'
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml

sudo systemctl enable --now containerd
sudo systemctl status containerd --no-pager
```

固定 crictl 使用与 kubelet 一致的 socket（消除默认探测多路 sock 的告警；路径以本机为准，常见为 /run/containerd/containerd.sock）：

```
sudo tee /etc/crictl.yaml <<'EOF'
runtime-endpoint: unix:///run/containerd/containerd.sock
image-endpoint: unix:///run/containerd/containerd.sock
EOF

sudo crictl version
```

2.4.1 kubeadm config images pull 报错 unknown service runtime.v1.ImageService

含义：**crictl / kubeadm 通过 CRI v1 访问镜像接口时，当前 containerd.sock 一侧未实现 ImageService**。常见于：

- **containerd 过旧**（需 **≥ 1.6**，Kubernetes 1.30 建议 **1.7.x**）；
- **/etc/containerd/config.toml 中禁用了 CRI**（disabled_plugins 含 cri 等）；
- 本机已装 **Docker CE**（存在 **docker0** 时多半已装），与发行版 **containerd** 并存，**实际连到的不是带完整 CRI 的 containerd**。

排查：

```
containerd --version
crictl version
sudo crictl info
grep -n disabled_plugins /etc/containerd/config.toml
ls -la /var/run/containerd/containerd.sock
```

处理（建议按顺序尝试）：

1. **升级 containerd**（Ubuntu 仓库）：
   sudo apt-get update && sudo apt-get install -y --only-upgrade containerd
   若版本仍偏旧，可改用 **Docker 官方仓库** 中的 **containerd.io**（与 kubeadm 文档常见做法一致），或安装与 1.30 匹配的较新 **containerd** 包。

2. **重建默认配置**（会覆盖 config.toml，若有自定义请先备份）：
   sudo sh -c 'containerd config default > /etc/containerd/config.toml'
   再执行 **sed** 打开 **SystemdCgroup = true**（同 **§2.4**），**确认未禁用 CRI**，然后：
   sudo systemctl restart containerd

3. **与 Docker 并存时**：明确 Kubernetes **只使用一个运行时**；要么卸载冲突包、只保留面向 K8s 的 **containerd**，要么改用 **cri-dockerd** 并让所有组件指向 **cri-docker.sock**（企业环境按需选型，此处从简）。

4. 再测：
   sudo crictl version
   sudo kubeadm config images pull --kubernetes-version=$(kubeadm version -o short) --image-repository=registry.aliyuncs.com/google_containers

2.4.2 containerd --version 为 v2.x，且 crictl 报 unknown service runtime.v1.RuntimeService

原因概要： containerd 2.x 与当前 **kubeadm 1.30 / crictl** 组合在部分安装方式下 **CRI 未正确对外暴露**，表现为 **RuntimeService / ImageService 均为 Unimplemented**。

推荐处理（生产向）： 降级并固定为 **containerd 1.7.x**（与 §2.4 一致），再重建配置。

```
sudo systemctl stop containerd
sudo apt-get update
apt-cache madison containerd | head -25
```

从列表中选取 **1.7.x** 的完整版本字符串（示例，**务必换成你机器 madison 输出中的真实版本**）：

```
# 将下方版本号替换为 madison 输出中完整的 1.7.x 一行（不可用通配符）
sudo apt-get install -y containerd=1.7.24-0ubuntu1~22.04.2
sudo apt-mark hold containerd

sudo cp -a /etc/containerd/config.toml /etc/containerd/config.toml.bak.$(date +%s) 2>/dev/null || true
sudo sh -c 'containerd config default > /etc/containerd/config.toml'
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml
sudo systemctl enable --now containerd
sudo systemctl restart containerd

containerd --version
sudo crictl version
```

若仓库中 **没有** 可用的 1.7，可改用 **Docker 官方 apt 源** 安装 **containerd.io 1.7**（与 Docker 文档一致），或换用企业内已验证的 **1.7** 离线包；**避免**在未经充分验证的节点上混用 **containerd 2.x + kubeadm 1.30**。

2.5 安装 kubeadm、kubelet、kubectl（1.30，清华 TUNA 源）

GPG 公钥（推荐，已验证可用）： 使用 **hkps://keyserver.ubuntu.com** 将签名密钥 **234654DA9A296436** 写入专用 keyring（与 pkgs.k8s.io / 国内镜像上的 InRelease 签名一致）。须安装 **dirmngr**，否则 --recv-keys 可能失败。

```
sudo apt-get update
sudo apt-get install -y apt-transport-https ca-certificates curl gpg gnupg dirmngr

sudo mkdir -p /etc/apt/keyrings
sudo rm -f /etc/apt/keyrings/kubernetes-apt-keyring.gpg

sudo gpg --no-default-keyring --keyring /etc/apt/keyrings/kubernetes-apt-keyring.gpg \
  --keyserver hkps://keyserver.ubuntu.com \
  --recv-keys 234654DA9A296436

sudo chmod 644 /etc/apt/keyrings/kubernetes-apt-keyring.gpg
```

确认 keyring 非空：

```
sudo gpg --no-default-keyring --keyring /etc/apt/keyrings/kubernetes-apt-keyring.gpg --list-keys --with-fingerprint
```

写入源列表（**signed-by 路径与上面 keyring 一致**）：

```
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://mirrors.tuna.tsinghua.edu.cn/kubernetes/core:/stable:/v1.30/deb/ /' | sudo tee /etc/apt/sources.list.d/kubernetes.list

sudo apt-get update
sudo apt-get install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl

kubeadm version
kubelet --version

sudo systemctl enable --now kubelet
```

备选：keyserver 不可用或 --recv-keys 失败时

1）换 80 端口 keyserver（部分网络仅放行 HTTP）：

```
sudo gpg --no-default-keyring --keyring /etc/apt/keyrings/kubernetes-apt-keyring.gpg \
  --keyserver hkp://keyserver.ubuntu.com:80 \
  --recv-keys 234654DA9A296436
```

2）用 Release.key 管道导入（任选能访问的地址）：

```
sudo rm -f /etc/apt/keyrings/kubernetes-apt-keyring.gpg
curl -fsSL https://mirrors.ustc.edu.cn/kubernetes/core:/stable:/v1.30/deb/Release.key | \
  sudo gpg --batch --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
sudo chmod 644 /etc/apt/keyrings/kubernetes-apt-keyring.gpg
```

或官方：

```
sudo rm -f /etc/apt/keyrings/kubernetes-apt-keyring.gpg
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.30/deb/Release.key | \
  sudo gpg --batch --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
sudo chmod 644 /etc/apt/keyrings/kubernetes-apt-keyring.gpg
```

3）仍报 NO_PUBKEY / not signed 时自检：

```
grep -R kubernetes /etc/apt/sources.list.d/
wc -c /etc/apt/keyrings/kubernetes-apt-keyring.gpg
sudo gpg --no-default-keyring --keyring /etc/apt/keyrings/kubernetes-apt-keyring.gpg --list-keys
```

4）排除镜像问题：deb 行可临时改为官方域名（keyring 不变）：

```
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.30/deb/ /' | sudo tee /etc/apt/sources.list.d/kubernetes.list
sudo apt-get update
```

3. 仅 MASTER（192.168.10.166）初始化集群

3.1 kubeadm init

版本号与已安装的 kubeadm 一致。**容器镜像**默认从 **registry.k8s.io** 拉取（会解析到海外仓库，大陆环境易出现 **i/o timeout / ImagePull**）。请使用阿里云 **google_containers** 作为替代仓库（与官方同步的常用做法）。

若此前 **kubeadm init 失败过半**，先清理再重做：

```
sudo kubeadm reset -f
```

建议先预拉取镜像（成功后再 init）：

```
sudo kubeadm config images pull \
  --kubernetes-version=$(kubeadm version -o short) \
  --image-repository=registry.aliyuncs.com/google_containers
```

初始化（务必带 image-repository）：

```
sudo kubeadm init \
  --kubernetes-version=$(kubeadm version -o short) \
  --image-repository=registry.aliyuncs.com/google_containers \
  --apiserver-advertise-address=192.168.10.166 \
  --control-plane-endpoint=192.168.10.166 \
  --pod-network-cidr=10.244.0.0/16
```

若 registry.aliyuncs.com/google_containers 仍超时，可换 **registry.cn-hangzhou.aliyuncs.com/google_containers** 再试；有企业内 Harbor 时改为 **内网同步同一批镜像的仓库地址**。

3.1.1 大陆网络必做：pause（Pod 沙箱）仍可能走 registry.k8s.io

即使 **kubeadm init 带了 --image-repository**，**kubelet** 创建静态 Pod 时仍可能去拉 **registry.k8s.io/pause:x.y**（日志里常见 **failed to get sandbox image**、europe-west3-docker.pkg.dev、i/o timeout），导致 **6443 无监听、控制面起不来**。

做法 A（推荐）：从国内源拉 pause，再 打标签 成 kubelet 期望的名字（版本以你机日志或下列命令为准）：

```
kubeadm config images list \
 --kubernetes-version=$(kubeadm version -o short) \
 --image-repository=registry.aliyuncs.com/google_containers | grep pause
```

版本号必须以 kubeadm config images list ... | grep pause 为准（Kubernetes **1.30.14** 一般为 **pause:3.9**）。若 **journalctl -u kubelet** 里仍出现 **pause:3.8**，多为旧配置残留，优先按 **3.9** 对齐；必要时 **3.8 / 3.9 各拉一次并分别 tag** 到 registry.k8s.io/pause:3.8 与 registry.k8s.io/pause:3.9。

以 **pause:3.9** 为例：

```
sudo ctr -n k8s.io images pull registry.aliyuncs.com/google_containers/pause:3.9
sudo ctr -n k8s.io images tag registry.aliyuncs.com/google_containers/pause:3.9 registry.k8s.io/pause:3.9
sudo systemctl restart kubelet
```

若 **ctr 提示命名空间**，可先 sudo crictl pull registry.aliyuncs.com/google_containers/pause:3.9，再用 **ctr** 完成 **tag**。

做法 B： 在 **/var/lib/kubelet/kubeadm-flags.env** 的 **KUBELET_KUBEADM_ARGS** 中增加（或合并进一行）：

```
--pod-infra-container-image=registry.aliyuncs.com/google_containers/pause:3.9
```

然后：

```
sudo systemctl daemon-reload
sudo systemctl restart kubelet
```

（版本号与 kubeadm config images list / 日志一致。）

所有工作节点在 kubeadm join 前，也应能拉到同一 **pause**（同样可用 **做法 A** 预拉并 **tag**，或 **做法 B** 写 kubelet 参数）。

3.1.2 排障：wait-control-plane / The API server is not healthy after 4m0s

含义：**kubelet 已起来**，但 **kube-apiserver 未在超时内就绪**（多为 **静态 Pod 内容器反复退出**、**etcd 未就绪**、**6443/2379 被占用**、**swap 未关**、**镜像不完整** 等）。

先不要立刻 reset，在 **Master** 上收集信息：

```
sudo systemctl status kubelet --no-pager
sudo journalctl -u kubelet -n 120 --no-pager
```

查看控制面容器（**务必指定与 kubelet 一致的 CRI socket**）：

```
sudo crictl --runtime-endpoint unix:///run/containerd/containerd.sock ps -a
sudo crictl --runtime-endpoint unix:///run/containerd/containerd.sock ps -a | grep -E 'etcd|kube-apiserver|kube-controller|kube-scheduler' || true
```

对 **Exited / 非 Running** 的容器取 **CONTAINER ID** 看日志（示例把 <ID> 换成实际 ID）：

```
sudo crictl --runtime-endpoint unix:///run/containerd/containerd.sock logs <ID>
```

也可看 **Pod 日志目录：**

```
sudo ls -la /var/log/pods/kube-system_* 2>/dev/null | tail
```

常见原因与处理：

现象 / 日志关键词	处理方向
ImagePullBackOff / manifest unknown	重新 **kubeadm config images pull**，确认 **--image-repository** 与 init 一致；必要时换仓库域名
address already in use :6443	sudo ss -lntp 查 **6443** 占用；关闭冲突进程后 **kubeadm reset -f** 再 init
etcd permission denied / data dir	检查 **/var/lib/etcd** 权限；reset 后重试
swap / cgroup 相关	确认 **§2.2** 已关 swap；**§2.4** SystemdCgroup = true
failed to get sandbox image / registry.k8s.io/pause / europe-west3-docker.pkg.dev 超时	见 **§3.1.1**：为 **pause** 从国内源拉取并 **tag**，或设置 **--pod-infra-container-image**
apiserver 连不上 etcd	先看 **etcd** 容器是否 Running，再看 **etcd** 日志

修复后若需重来：

```
sudo kubeadm reset -f
```

再执行 **§3.1** 的 **config images pull + kubeadm init**。

3.2 管理员 kubeconfig

```
mkdir -p "$HOME/.kube"
sudo cp -i /etc/kubernetes/admin.conf "$HOME/.kube/config"
sudo chown "$(id -u):$(id -g)" "$HOME/.kube/config"
```

3.3 安装 Calico（v3.28.2，大陆优先用阿里云镜像）

官方清单里的镜像来自 **docker.io/calico/...**，在大陆常需 **阿里云 ACR 同步** 或 **containerd 对 docker.io 配置镜像加速**。任选其一即可。

方式 A（推荐）：全节点为 docker.io 配置镜像加速，YAML 不改

在 **Master 与所有 Worker** 的 **containerd** 上为 **docker.io** 配置加速（地址以 **阿里云控制台 → 容器镜像服务 ACR → 镜像加速器** 中分配的为准，每人不同；亦可使用单位统一提供的 **registry 加速域名**）。

示例（**把加速地址换成你自己的**；containerd ≥ 1.4 支持 config_path）：

```
sudo mkdir -p /etc/containerd/certs.d/docker.io
sudo tee /etc/containerd/certs.d/docker.io/hosts.toml <<'EOF'
server = "https://registry-1.docker.io"
[host."https://<你的阿里云镜像加速器 ID>.mirror.aliyuncs.com"]
```

```
  capabilities = ["pull", "resolve"]
EOF
```

确认 **/etc/containerd/config.toml** 中已启用 **config_path = "/etc/containerd/certs.d"**（若使用 containerd config default 生成，一般已包含；否则在 [plugins."io.containerd.grpc.v1.cri"] 下增加该字段），然后：

```
sudo systemctl restart containerd
```

再 **仅在 Master**（能执行 kubectl 的节点）安装 Calico：

```
kubectl apply -f https://mirror.ghproxy.com/https://raw.githubusercontent.com/projectcalico/calico/v3.28.2/manifests/calico.yaml
```

或直接（能访问 GitHub 时）：

```
kubectl apply -f https://raw.githubusercontent.com/projectcalico/calico/v3.28.2/manifests/calico.yaml
```

方式 B：将 Calico 镜像同步到阿里云 ACR，再改 YAML 中的镜像前缀

1. 在 **容器镜像服务 ACR** 创建实例与命名空间（如 **k8s**），使用 **镜像同步** 或 **docker pull + docker tag + docker push**，将 **calico/node**、**calico/cni**、**calico/kube-controllers** 等（版本 **v3.28.2**）同步到例如：
   registry.cn-hangzhou.aliyuncs.com/<你的命名空间>/
   （具体域名以控制台为准，华北/华东地域域名可能不同。）
2. 下载清单并替换前缀。

重要： registry.cn-hangzhou.aliyuncs.com/k8s/calico/... 仅为 **示例**。你必须已在 ACR 中 **真实存在** 对应仓库并已 **推送/同步** 镜像；否则会报 **pull access denied / insufficient_scope / repository does not exist**。**禁止**在未推送镜像时直接 sed 成示例地址。

将下面命令中的 **YOUR_ACR_REGISTRY** 换成你在控制台看到的完整前缀（含命名空间），例如 registry.cn-hangzhou.aliyuncs.com/mycompany：

```
curl -fsSL -o /tmp/calico.yaml \
  https://raw.githubusercontent.com/projectcalico/calico/v3.28.2/manifests/calico.yaml

grep -E '^\s*image:\s' /tmp/calico.yaml | sort -u

sudo sed -i 's#docker.io/calico/#YOUR_ACR_REGISTRY/calico/#g' /tmp/calico.yaml
```

若清单里镜像 **不带** docker.io/ 前缀而写作 **calico/xxx**，可改为：

```
sudo sed -i 's#^\(\s*image:\s*\)calico/#\1YOUR_ACR_REGISTRY/calico/#g' /tmp/calico.yaml
```

推送示例（在任意能访问 Docker Hub 的机器上执行后，再 **docker push** 到你的 ACR）：

```
docker pull docker.io/calico/cni:v3.28.2
docker tag docker.io/calico/cni:v3.28.2 YOUR_ACR_REGISTRY/calico/cni:v3.28.2
docker login YOUR_ACR_REGISTRY
docker push YOUR_ACR_REGISTRY/calico/cni:v3.28.2
```

对 **calico/node**、**calico/kube-controllers** 等重复同样操作（镜像名以 grep image 为准）。

3. 若 ACR 为 **私有库**，需在同一命名空间创建 **imagePullSecrets** 并在 Calico DaemonSet/Deployment 中引用（略，按 ACR 文档操作）。

4. 应用：

```
kubectl apply -f /tmp/calico.yaml
```

方式 C：华为云 SWR（ddn-k8s）固定镜像 — 替换为下列三条地址

若使用华为云 **软件仓库 SWR** 已提供的镜像（**华北-北京四**示例如下），下载官方清单后 **按组件分别替换**（版本 **v3.28.2** 须与清单一致）：

组件	替换为
cni	swr.cn-north-4.myhuaweicloud.com/ddn-k8s/docker.io/calico/cni:v3.28.2
kube-controllers	swr.cn-north-4.myhuaweicloud.com/ddn-k8s/docker.io/calico/kube-controllers:v3.28.2
node	swr.cn-north-4.myhuaweicloud.com/ddn-k8s/docker.io/calico/node:v3.28.2

注意： **方式 C** 必须在 **未经过方式 B 错误示例** 的 **干净** calico.yaml 上只做 **一轮** sed。若曾把镜像改成 registry.cn-hangzhou.aliyuncs.com/k8s/calico/... 再执行 SWR 替换，会出现 **registry.cn-hangzhou.aliyuncs.com/k8s/swr.cn-north-4...** 这种 **双重前缀**。修复任选其一：**重新 curl 官方 YAML 后仅执行下方 sed**，或执行：

```
sudo sed -i 's|registry.cn-hangzhou.aliyuncs.com/k8s/||g' /tmp/calico.yaml

curl -fsSL -o /tmp/calico.yaml \
  https://raw.githubusercontent.com/projectcalico/calico/v3.28.2/manifests/calico.yaml

grep -E 'image:.*calico' /tmp/calico.yaml | sort -u
```

若清单中为 **docker.io/calico/...** 写法，执行：

```
sudo sed -i \
  -e 's|docker.io/calico/cni:v3.28.2|swr.cn-north-4.myhuaweicloud.com/ddn-k8s/docker.io/calico/cni:v3.28.2|g' \
  -e 's|docker.io/calico/kube-controllers:v3.28.2|swr.cn-north-4.myhuaweicloud.com/ddn-k8s/docker.io/calico/kube-controllers:v3.28.2|g' \
  -e 's|docker.io/calico/node:v3.28.2|swr.cn-north-4.myhuaweicloud.com/ddn-k8s/docker.io/calico/node:v3.28.2|g' \
  /tmp/calico.yaml
```

若清单中 **无** docker.io/ 前缀（写作 **calico/cni:v3.28.2**），可改为：

```
sudo sed -i \
  -e 's|calico/cni:v3.28.2|swr.cn-north-4.myhuaweicloud.com/ddn-k8s/docker.io/calico/cni:v3.28.2|g' \
  -e 's|calico/kube-controllers:v3.28.2|swr.cn-north-4.myhuaweicloud.com/ddn-k8s/docker.io/calico/kube-controllers:v3.28.2|g' \
  -e 's|calico/node:v3.28.2|swr.cn-north-4.myhuaweicloud.com/ddn-k8s/docker.io/calico/node:v3.28.2|g' \
  /tmp/calico.yaml
```

再次检查是否还有 **docker.io/calico** 或其它 **calico/*** 未替换（例如 **pod2daemon-flexvol**）；若有，需在 SWR 存在对应镜像或改回 **方式 A** 走 docker.io 加速：

```
grep -E 'image:.*calico' /tmp/calico.yaml

kubectl apply -f /tmp/calico.yaml
```

说明： SWR 若需登录，请在各节点 **docker login swr.cn-north-4.myhuaweicloud.com** 或在集群中配置 **imagePullSecrets**（按华为云文档）。

安装后检查

```
kubectl get pods -n kube-system
kubectl wait --for=condition=ready pod -l k8s-app=calico-node -n kube-system --timeout=300s || \
  kubectl get pods -n kube-system -l k8s-app=calico-node
```

3.4 生成 Worker 加入命令

```
kubeadm token create --print-join-command
```

将输出的整行 kubeadm join … 在 **192.168.11.116** 与 **192.168.11.160** 上以 **sudo** 执行。

3.5 Master 上确认节点

```
kubectl get nodes -o wide
```

4. Worker 节点（192.168.11.116、192.168.11.160）

kubeadm join **没有** --image-repository 参数；若只用默认 join 命令，Worker 仍会向 **registry.k8s.io** 拉 **kube-proxy**、**pause** 等，在大陆易 **超时**。请在每台 Worker 上使用 **配置文件**（与 Master 使用 **同一 imageRepository**）。

将 **3.4** 里 kubeadm token create --print-join-command 输出中的 **token**、**sha256** 填入下面模板，保存为例如 /root/join.yaml：

```yaml
apiVersion: kubeadm.k8s.io/v1beta3
kind: JoinConfiguration
discovery:
  bootstrapToken:
    apiServerEndpoint: 192.168.10.166:6443
    token: "<粘贴 token>"
    caCertHashes:
      - "sha256:<粘贴 hash>"
nodeRegistration:
  criSocket: unix:///var/run/containerd/containerd.sock
---
apiVersion: kubeadm.k8s.io/v1beta3
kind: ClusterConfiguration
imageRepository: registry.aliyuncs.com/google_containers
```

执行：

```
sudo kubeadm join --config=/root/join.yaml
```

注意： 使用 --config 时不要再混用同一 join 的其它 discovery 参数；criSocket 若与你机上的 containerd 路径不一致请改为实际路径（常见为 unix:///var/run/containerd/containerd.sock）。

若你希望继续用 **一行式** kubeadm join 192.168.10.166:6443 --token ...，则须在 **所有节点** 的 **containerd** 上为 registry.k8s.io 配置可达的 **镜像加速/镜像仓库**（由运维统一维护，此处不展开）。

若 token 过期，在 Master 执行：

```
kubeadm token create --print-join-command
```

5. NFS 服务端（仅 192.168.11.160）

5.1 安装服务

必须先执行本节；若未安装就出现 exportfs: command not found 或 Unit file nfs-server.service does not exist，说明 **nfs-kernel-server 尚未装上**（exportfs 与该 systemd 单元均由本包提供）。

```
sudo apt-get update
sudo apt-get install -y nfs-kernel-server
```

5.2 导出目录与 /etc/exports

```
sudo mkdir -p /export/k8s-data
sudo chown nobody:nogroup /export/k8s-data
sudo chmod 755 /export/k8s-data

sudo tee /etc/exports <<'EOF'
/export/k8s-data 192.168.10.0/24(rw,sync,no_subtree_check,no_root_squash)
/export/k8s-data 192.168.11.0/24(rw,sync,no_subtree_check,no_root_squash)
EOF

sudo exportfs -rav
sudo systemctl enable --now nfs-server
showmount -e localhost
```

5.3 各节点安装 NFS 客户端（三台）

```
sudo apt-get update
sudo apt-get install -y nfs-common
```

5.4 验证（任意节点）

```
showmount -e 192.168.11.160
```

6. 集群内 NFS 动态供给（NFS-SUBDIR-EXTERNAL-PROVISIONER，HELM）

在 **已配置 kubectl** 的机器上执行。

6.1 安装 Helm 3（华为云镜像）

```
curl -fsSL -o /tmp/helm.tar.gz https://mirrors.huaweicloud.com/helm/v3.14.4/helm-v3.14.4-linux-amd64.tar.gz
sudo tar -xzf /tmp/helm.tar.gz -C /tmp
sudo mv /tmp/linux-amd64/helm /usr/local/bin/helm
helm version
```

若需其他版本，将 URL 中 v3.14.4 改为目标版本（路径规则相同）。

6.2 Helm Chart 仓库（需能访问 GitHub Pages）

```
helm repo add nfs-subdir-external-provisioner https://kubernetes-sigs.github.io/nfs-subdir-external-provisioner/
helm repo update
```

若 **helm repo update** 失败，见 **第 7.0 节**。

6.3 安装 Provisioner（StorageClass 名 nfs-client）

```
kubectl create namespace nfs-provisioner --dry-run=client -o yaml | kubectl apply -f -

helm upgrade --install nfs-subdir-external-provisioner nfs-subdir-external-provisioner/nfs-subdir-external-provisioner \
  --namespace nfs-provisioner \
  --set nfs.server=192.168.11.160 \
  --set nfs.path=/export/k8s-data \
  --set storageClass.name=nfs-client \
  --set storageClass.defaultClass=true
```

默认镜像为 **registry.k8s.io/sig-storage/nfs-subdir-external-provisioner:v4.0.2**，在大陆常 **拉取超时**，见 **§6.4**。

```
kubectl get pods -n nfs-provisioner
kubectl get storageclass
```

6.4 大陆网络：覆盖 Provisioner 镜像（registry.k8s.io 拉取超时）

Chart 默认 **image.repository / image.tag** 指向 **registry.k8s.io**。若 Pod 事件为 **Failed to pull image ... europe-west3-docker.pkg.dev ... i/o timeout**，需改为 **你可访问的仓库**（与 Calico 相同，可使用 **华为 SWR ddn-k8s** 前缀；**路径须与控制台实际镜像一致**，不存在时需先 **pull + push** 同步）。

查看 Chart 默认值：

```
helm show values nfs-subdir-external-provisioner/nfs-subdir-external-provisioner | grep -A10 '^image:'
```

完整镜像地址示例（与 **registry.k8s.io** 路径对齐的 SWR 同步）：

swr.cn-north-4.myhuaweicloud.com/ddn-k8s/registry.k8s.io/sig-storage/nfs-subdir-external-provisioner:v4.0.2

Helm 需拆成 **repository**（无 **tag**） + **tag**：

字段	值
image.repository	swr.cn-north-4.myhuaweicloud.com/ddn-k8s/registry.k8s.io/sig-storage/nfs-subdir-external-provisioner
image.tag	v4.0.2

已安装后 仅改镜像：

```
helm upgrade nfs-subdir-external-provisioner nfs-subdir-external-provisioner/nfs-subdir-external-provisioner \
  --namespace nfs-provisioner \
  --reuse-values \
  --set image.repository=swr.cn-north-4.myhuaweicloud.com/ddn-k8s/registry.k8s.io/sig-storage/nfs-subdir-external-provisioner \
  --set image.tag=v4.0.2
```

若 SWR 上 **无** 该路径，可在能访问 **registry.k8s.io** 的环境执行 **docker pull / docker tag / docker push** 推到你的 SWR，再把 **image.repository** 指到 **自有仓库地址**。

新安装时 可合并进一条 helm upgrade --install（在 **§6.3** 的 --set 后追加）：

```
--set image.repository=swr.cn-north-4.myhuaweicloud.com/ddn-k8s/registry.k8s.io/sig-storage/nfs-subdir-external-provisioner \
--set image.tag=v4.0.2
```

7. KUBE-PROMETHEUS-STACK（HELM）

7.0 Chart 仓库访问失败时（国内常见）

任选其一：

1. **为当前终端配置 HTTP/HTTPS 代理** 后再执行 helm repo add / helm repo update。
2. 在可访问 github.io 的机器上执行 helm pull prometheus-community/kube-prometheus-stack --version X.Y.Z，将 .tgz 拷入内网后：
   helm upgrade --install kube-prometheus-stack ./kube-prometheus-stack-X.Y.Z.tgz -n monitoring.
3. **helm 报 unexpected EOF（从 GitHub Releases 下载 kube-prometheus-stack-*.tgz 中断）**：多为访问 GitHub / release-assets.githubusercontent.com 不稳定。公共前缀镜像（如 **mirror.ghproxy.com** 等）**经常失效**，不要只依赖某一种。

   优先顺序建议：

   - **有 HTTP/HTTPS 代理时**（最稳）：export https_proxy=http://IP:端口（及需要时的 http_proxy），再 **curl** 或 **helm repo update / helm pull**。
   - **无代理时**：对 **官方 Release 直链** 用 **curl -L 多重重试**；失败则用 **wget -c** 断点续传，或换能访问 GitHub 的机器下载后 **scp** 到 **ubuntu-4097**。

   手动下载再安装（将 **83.4.0** 换成你报错 URL 里的版本号；直链为 GitHub Releases 标准地址）：

```
CHART_VER=83.4.0
URL="https://github.com/prometheus-community/helm-charts/releases/download/kube-prometheus-stack-${CHART_VER}/kube-prometheus-stack-${CHART_VER}.tgz"
OUT="/tmp/kube-prometheus-stack-${CHART_VER}.tgz"

curl -fL --retry 5 --retry-delay 3 --connect-timeout 30 -o "$OUT" "$URL" \
  || wget -c -O "$OUT" "$URL"

ls -lh "$OUT"
file "$OUT"   # 应为gzip compressed data，勿为HTML 错误页

helm upgrade --install kube-prometheus-stack "$OUT" \
  --namespace monitoring \
  --create-namespace \
  -f /tmp/values-monitoring.yaml \
  --set prometheus.prometheusSpec.retention=15d
```

若仍要试 **第三方 GitHub 加速前缀**，自行搜索当前可用的 **ghproxy / GitHub 加速** 域名，把 **https://github.com/...tgz** 整段 URL 接到该服务要求的格式后下载；**不保证长期可用**。下载后务必 **file / tar tzf** 确认是有效 **tgz**，再执行 **helm upgrade --install ... /path/to.tgz**。

7.1 添加仓库并安装

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update

kubectl create namespace monitoring --dry-run=client -o yaml | kubectl apply -f -

helm search repo kube-prometheus-stack --versions | head
```

快速验证（数据不持久）：

```
helm upgrade --install kube-prometheus-stack prometheus-community/kube-prometheus-stack \
  --namespace monitoring \
  --set prometheus.prometheusSpec.retention=15d \
  --set prometheusOperator.admissionWebhooks.enabled=true
```

指定 Chart 版本（将 X.Y.Z 替换为上一步 helm search 第二列真实版本号）：

```
helm upgrade --install kube-prometheus-stack prometheus-community/kube-prometheus-stack \
  --namespace monitoring \
  --version X.Y.Z \
  --set prometheus.prometheusSpec.retention=15d \
  --set prometheusOperator.admissionWebhooks.enabled=true
```

7.2 使用 NFS StorageClass 持久化（nfs-client 已存在时）

本仓库根目录提供同名示例文件 **values-monitoring.yaml**（与下述内容一致），可 **scp** 到节点后直接 **helm upgrade -f values-monitoring.yaml**。

```
cat <<'YAML' | tee /tmp/values-monitoring.yaml
prometheus:
```

```yaml
  prometheusSpec:
    retention: 15d
    storageSpec:
      volumeClaimTemplate:
        spec:
          storageClassName: nfs-client
          accessModes: ["ReadWriteOnce"]
          resources:
            requests:
              storage: 50Gi
  service:
    type: NodePort
    nodePort: 30090

alertmanager:
  alertmanagerSpec:
    storage:
      volumeClaimTemplate:
        spec:
          storageClassName: nfs-client
          accessModes: ["ReadWriteOnce"]
          resources:
            requests:
              storage: 70Gi
  service:
    type: NodePort
    nodePort: 30903

grafana:
  persistence:
    enabled: true
    storageClassName: nfs-client
    size: 70Gi
  # Grafana 持久化卷的 init 容器；若曾使用华为 SWR 上的 Rancher 同步镜像，例如
  # swr.cn-north-4.myhuaweicloud.com/ddn-k8s/docker.io/rancher/mirrored-library-busybox:1.37.0
  # 可改为 Docker Hub 等价镜像；repository 只写 library/busybox（勿写 docker.io/ 前缀，否则会变成 docker.io/docker.io/...）
  initChownData:
    image:
      repository: library/busybox
      tag: "1.37.0"
  service:
    type: NodePort
    nodePort: 30300
```

YAML

```bash
helm upgrade --install kube-prometheus-stack prometheus-community/kube-prometheus-stack \
  --namespace monitoring \
```

```
 -f /tmp/values-monitoring.yaml \
 --set prometheus.prometheusSpec.retention=15d
```

上例已合并 **NFS 持久化** 与 **NodePort 对外访问**（**30300 / 30090 / 30903**，见 **§7.3**）。若仅需集群内 **ClusterIP**，删除 **prometheus / alertmanager / grafana** 下各自的 **service:** 段即可。

Grafana init 镜像拉取 docker.io/docker.io/library/busybox：
grafana.initChownData.image.repository 应只写 **library/busybox**（**不要**写 **docker.io/library/busybox**），否则子 Chart 会再拼 **docker.io/**，导致重复前缀与 **ErrImagePull**。

仅改 busybox、不改 values 文件时（与上面 YAML 等效）：

```
helm upgrade kube-prometheus-stack prometheus-community/kube-prometheus-stack \
 --namespace monitoring \
 -f /tmp/values-monitoring.yaml \
 --set grafana.initChownData.image.repository=library/busybox \
 --set grafana.initChownData.image.tag=1.37.0
```

7.3 检查与访问 Grafana

```
kubectl get pods -n monitoring
kubectl get svc -n monitoring

kubectl port-forward -n monitoring svc/kube-prometheus-stack-grafana 3000:80

kubectl get secret -n monitoring kube-prometheus-stack-grafana \
 -o jsonpath="{.data.admin-password}" | base64 -d
echo
```

浏览器：http://127.0.0.1:3000（用户一般为 admin）。

在页面中添加 Prometheus 数据源： 登录 Grafana 后（不同版本菜单位置略有差异，以下以常见布局为准）：

1. 左侧菜单打开 **Connections**（或 **Configuration**）→ **Data sources** → **Add data source**。
2. 选择 **Prometheus**。
3. **Name**：可填 **Prometheus**（或任意名称）。
4. **Prometheus server URL**（必填）：填写集群内 Service 地址，**不要**使用占位符 **http://localhost:9090**（Grafana 在 Pod 内，localhost 不是 Prometheus）。默认 **kube-prometheus-stack**、命名空间 **monitoring** 时一般为：

http://kube-prometheus-stack-prometheus.monitoring.svc.cluster.local:9090

若短名可解析，也可用 **http://kube-prometheus-stack-prometheus.monitoring:9090**。端口以 **kubectl get svc -n monitoring kube-prometheus-stack-prometheus** 为准（多为 **9090**）。
5. **Authentication**：未对 Prometheus 启用独立认证时保持默认即可。
6. 点击 **Save & test**，应显示连接成功。

说明： Helm 安装后 **Data sources** 里可能**已有**由 chart 注入的 Prometheus，可先打开查看是否可用，避免重复添加。若 Service 名因 **release 名**不同而变化，以 **kubectl get svc -n monitoring** 列名为准替换 URL 中的主机名。

集群外访问（不经过本机 port-forward）： 当前 **kube-prometheus-stack-grafana** 等为 **ClusterIP**，集群外默认不可直达。任选其一（按环境从简到繁）：

1. **NodePort（无 Ingress、内网直达）**
   含义： 将 **Service** 暴露为 **type: NodePort**，Kubernetes 会在每个节点上监听同一**高位端口**（默认范围 **30000–32767**），集群外浏览器访问 **http://<任意节点 IP>:<nodePort>** 即可（无需经过 Ingress）。
   注意： **nodePort 全集群唯一**；若端口被占用，helm upgrade 会失败，换一个未占用的即可。节点 **防火墙 / 安全组** 需放行对应 **TCP**。

   §7.2 中 **/tmp/values-monitoring.yaml** 已含与下文一致的 **NodePort** 片段；若尚未使用 §7.2 文件，可在任意 **values-monitoring.yaml** 里为 **grafana / prometheus / alertmanager** 追加同级 **service:**（仅 Grafana 时只加 **grafana.service** 亦可）：

```yaml
grafana:
  service:
    type: NodePort
    nodePort: 30300

prometheus:
  service:
    type: NodePort
    nodePort: 30090

alertmanager:
  service:
    type: NodePort
    nodePort: 30903
```

应用配置：

```bash
helm upgrade kube-prometheus-stack prometheus-community/kube-prometheus-stack \
  --namespace monitoring \
  -f /tmp/values-monitoring.yaml \
  --reuse-values
```

（若使用本地 **.tgz** chart，将 **prometheus-community/kube-prometheus-stack** 换成 **./kube-prometheus-stack-X.Y.Z.tgz**。）

验证与访问：

```bash
kubectl get svc -n monitoring | grep -E 'grafana|prometheus|alertmanager'
```

期望 **TYPE 为 NodePort**，**PORT(S)** 中含 **30300:80/TCP** 等形式。浏览器示例：**http://192.168.x.x:30300**（Grafana）、**:30090**（Prometheus）、**:30903**（Alertmanager）。
Grafana 账号一般为 **admin**，密码见 **§7.3** 中 **kubectl get secret ...**。
若某组件 Chart 键名与上表不一致，以 **helm show values prometheus-community/kube-prometheus-stack | grep -A20 '^grafana:'** 等为准，重点找各组件下的 **service:** 块。

2. **Ingress（有 Ingress Controller 时推荐）**
   在 **grafana**（及可选的 **prometheus / alertmanager**）下启用 **ingress.enabled: true**，配置

hosts、**ingressClassName**（如 **nginx**）、**tls**。浏览器用域名访问；**Grafana 默认 Service 端口仍为 80**（Ingress 后端指向 **kube-prometheus-stack-grafana:80**）。

3. **LoadBalancer**
   若集群已对接云 LB 或 **MetalLB** 等，将对应组件的 **service.type: LoadBalancer**，用分配的 **EXTERNAL-IP** 访问。

安全： 对外暴露前务必改强 **Grafana admin 密码**（Chart 的 **grafana.adminPassword** 或 Secret），并限制 **防火墙 / 安全组** 仅允许管理网段；生产环境优先 **HTTPS + Ingress**。

7.4 无在线 Helm 仓库时（仅本地包）

```
helm template kube-prometheus-stack prometheus-community/kube-prometheus-stack \
 --namespace monitoring \
 -f /tmp/values-monitoring.yaml \
 > /tmp/kube-prometheus-stack-rendered.yaml

kubectl apply -f /tmp/kube-prometheus-stack-rendered.yaml
```

（需已能通过 helm pull 取得 chart 包并解压到本地，或已有 values-monitoring.yaml 与 chart 路径。）

---

8. A100 + MIG + GPU Operator（后续）

8.0 Kubernetes 如何对接 NVIDIA（总览）

对接指：**工作负载 Pod 能在指定节点上访问 GPU**，并在调度时声明 **nvidia.com/* 资源**。整体分三层（由下至上）：

层次	做什么	常见做法
1. 节点操作系统	安装 **NVIDIA 驱动**，/dev/nvidia* 可用	**apt 装 nvidia-driver-XXX**（或与发行版匹配的驱动包）；**nvidia-smi** 在节点上正常
2. 容器运行时	让 **containerd** 能把 GPU 设备/库挂进容器	安装 **NVIDIA Container Toolkit**，执行 **nvidia-ctk runtime configure --runtime=containerd** 并 **systemctl restart containerd**（见 **§8.1**）
3. 集群组件	让 **kubelet** 上报可分配 GPU，**调度器** 按申请分配	**二选一：** ① **NVIDIA Device Plugin**（DaemonSet，较简）；② **NVIDIA GPU Operator**（Helm，顺带管理驱动插件、device-plugin、DCGM 等，见 **§8.3**）

工作负载侧： 在 Pod 里 **resources.limits** 申请 GPU，例如整卡：

```yaml
resources:
 limits:
  nvidia.com/gpu: "1"
 requests:
  nvidia.com/gpu: "1"
```

镜像需带 **CUDA 运行时**（或与驱动匹配的 **nvidia/cuda** 系镜像）。**MIG** 场景下资源名可能是 **nvidia.com/mig-...**（与 profile、device-plugin 配置一致），需在节点上先 **nvidia-smi -mig 1** 并建好 **GI/CI**（见 **§8.2**），再以 GPU Operator / device-plugin 文档为准配置。

验证命令（集群侧）：

```
kubectl describe node <gpu 节点名> | grep -A2 "Allocatable:"
# 应能看到 nvidia.com/gpu 或 nvidia.com/mig-*（取决于插件与 MIG）

kubectl describe node <gpu 节点名> | grep -i nvidia
kubectl get pods -n kube-system -l name=nvidia-device-plugin-daemonset
# 若用 GPU Operator：kubectl get pods -n gpu-operator
```

小结： 先有 **驱动 + containerd 集成**，再装 **device-plugin 或 GPU Operator**，Pod 里 **limits.nvidia.com/gpu**（或 MIG 资源）即可与 NVIDIA 对接。

若 Master 与 Worker 均带 GPU： 每一台都要按 **§8.1** 装 **驱动**、配置 **containerd + NVIDIA 运行时**（与 Worker 相同）；**nvidia.com/mig.config** 亦 **按节点单独打标签**。GPU Operator 各组件 DaemonSet 一般带 **控制面污点容忍**，可调度到 Master。生产环境常 **不让业务 GPU Pod 上控制面**（保留 **node-role.kubernetes.io/control-plane:NoSchedule**，或不暴露 Master 的 **nvidia.com/gpu**）；若确需在 Master 跑 GPU，需为工作负载配置 **容忍** 并接受 **与 apiserver 争用资源** 的风险。

8.1 apt 出现 Key is stored in legacy trusted.gpg（NVIDIA 源）
含义：这是 **警告（W:）**，不是失败。原因是早年用 **apt-key add** 把密钥写进了 **/etc/apt/trusted.gpg**；Ubuntu 建议改为 **/usr/share/keyrings/*.gpg + deb [signed-by=...]**。你日志里的 **ubuntu18.04** 源多为旧文档或旧脚本遗留，应换成 **当前官方 unified deb 源**。

在 **GPU 节点**上按顺序执行（可先备份 sources.list.d）：

```
sudo mkdir -p /root/apt-backup-nvidia
sudo cp -a /etc/apt/sources.list.d /root/apt-backup-nvidia/

# 删除常见的旧 NVIDIA 源列表（没有则忽略报错）
sudo rm -f /etc/apt/sources.list.d/nvidia-docker.list
sudo rm -f /etc/apt/sources.list.d/nvidia-container-runtime.list
sudo rm -f /etc/apt/sources.list.d/libnvidia-container.list
sudo rm -f /etc/apt/sources.list.d/nvidia-container-toolkit.list
```

写入 **官方推荐** 的 keyring 与单一仓库（与 NVIDIA Container Toolkit 安装文档 一致）：

```
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey | \
  sudo gpg --dearmor -o /usr/share/keyrings/nvidia-container-toolkit-keyring.gpg

curl -fsSL https://nvidia.github.io/libnvidia-container/stable/deb/nvidia-container-toolkit.list | \
  sed 's#deb https://#deb [signed-by=/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg] https://#g' | \
  sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list

sudo apt-get update
```

若仍有与 NVIDIA 相关的 **legacy** 提示，可列出并删除旧密钥（**谨慎**，只删 NVIDIA 条目）：

```
sudo apt-key list 2>/dev/null | grep -i nvidia -B1 -A1
# 记下 nvidia 仓库密钥的 Key id，例如末尾 8 位 ABCDE123，然后：
# sudo apt-key del ABCDE123
```

需要在本机使用 GPU 容器 / 配置 containerd 时再安装工具包：

```
sudo apt-get update
sudo apt-get install -y nvidia-container-toolkit
sudo nvidia-ctk runtime configure --runtime=containerd
sudo systemctl restart containerd
```

若 **nvidia-ctk runtime configure --runtime=containerd** 报错 **unrecognized runtime 'containerd'**：表示当前 **nvidia-container-toolkit 为 1.13.x**，该版本的 **nvidia-ctk** 尚不支持用一条命令配置 **containerd**（**1.14.0+** 才支持）。任选其一：

1. **升级 toolkit（推荐）：** sudo apt-cache madison nvidia-container-toolkit 查看仓库是否有 **1.14.x**，若有则 **sudo apt-get install -y nvidia-container-toolkit**（或指定新版本）后再执行 **nvidia-ctk runtime configure --runtime=containerd**。
2. **手动改 containerd：** 按 NVIDIA Container Toolkit 文档 — containerd / Kubernetes 在 **/etc/containerd/config.toml** 中增加 **nvidia** runtime 段（**runtime_type 须与本机 containerd 一致**，常见为 **io.containerd.runc.v2**），保存后 **sudo systemctl restart containerd**。

8.2 节点上启用 MIG（示例）

节点已安装 NVIDIA 驱动后，**先保证 GPU 上无任何进程**（含显示、CUDA、监控、K8s 已调度到本机的 GPU Pod）。否则会出现 **pending enable state / In use by another client**，MIG 不会真正生效。

典型占用： nvidia-smi 里 **Processes** 出现 **/usr/lib/xorg/Xorg**（桌面环境）、**nvidia-persistenced**、或业务进程。处理思路：

1. **带桌面的节点：**切换到纯文本多用户（或临时停显示服务），再启用 MIG：

```
# 示例：使用 systemd 时停图形界面（发行版不同，服务名可能是 gdm3 / sddm / lightdm）
sudo systemctl isolate multi-user.target
# 或：sudo systemctl stop gdm3

sudo nvidia-smi
# 确认 Processes 为空后：
sudo nvidia-smi -mig 1
sudo reboot   # 若提示需重启才能生效，务必重启后再 nvidia-smi -mig 1 / mig -lgip
```

2. **K8s GPU 节点：**先 **kubectl cordon + 驱逐/删掉** 使用该 GPU 的 Pod，必要时停 **nvidia-device-plugin** / GPU Operator 相关组件，再执行 **nvidia-smi -mig 1**。
3. 仍报占用时：**重启进入无桌面会话**后再执行，或 **sudo lsof /dev/nvidia0** / **sudo fuser -v /dev/nvidia*** 查进程并结束。

4. **Processes 已为 No running processes found 仍报 In use by another client / pending enable**（你当前这类情况）：多为 **上次未完成的 MIG 切换** 或 **守护进程持有设备**。按顺序尝试：
   - **先重启**（sudo reboot）。重启后 **用 SSH 登录**，不要切回桌面；必要时 **sudo systemctl disable gdm3**（或 **sddm / lightdm**）后重启一次，确保开机不自动起图形界面，再执行 **sudo nvidia-smi -mig 1**。
   - 临时停 **持久化守护进程** 再试（无关键 GPU 业务时）：

```
sudo systemctl stop nvidia-persistenced
sudo nvidia-smi -mig 1
# 成功后如需可再：sudo systemctl start nvidia-persistenced
```

- 仍失败时查阅 **dmesg | tail -50 / journalctl -u nvidia-***；部分环境需 NVIDIA 文档中的 **GPU reset** 或 **卸载重装驱动前备份**——生产环境以厂商文档为准。

启用成功后校验（nvidia-smi 顶部应出现 **MIG** 相关信息，具体以驱动输出为准）：

```
nvidia-smi
nvidia-smi mig -lgip
```

说明：

- **Enabled MIG Mode** 仅表示 **硬件 MIG 已打开**；若 **MIG devices 仍为 No MIG devices found**，属正常——还需按业务 **创建 GPU Instance（GI）**，必要时再建 **Compute Instance（CI）**（见 **nvidia-smi mig -h**、NVIDIA *MIG User Guide*）。
- 若出现 **persistence mode is disabled** 类提示：MIG 与 **Persistence Mode** 存在已知限制（见 **man nvidia-smi** *Known Issues*），一般可忽略；需持久化时再按手册与场景评估。
- **nvidia-smi -mig 1** 在 **已启用** 时再次执行只会重复提示 **Enabled**，属正常；**是否已有可调度切片** 以 **nvidia-smi mig -lgip**（profile 余量）与 **nvidia-smi -L / nvidia-smi mig -lgi** 为准。

nvidia-smi mig -lgip 表头含义（A100 40GB 示例）： 第二列 **Name** 为 profile 名；**ID** 为 **nvidia-smi mig -cgi** 使用的 profile 编号；**Free/Total** 为该 profile 在当前 GPU 上还能建几个 / 理论最大几个；**Memory** 为该 GI 显存规格（GiB）。

常见需求	Profile（示例名）	表中 **ID**（以你机 -lgip 为准）
整卡一切片（约 40GB 显存给单任务）	7g.40gb	0
中等切分	3g.20gb / 2g.10gb 等	以 -lgip 为准
多租小实例	1g.5gb（最多 7 个）	19

创建 GPU Instance 示例（单 GPU、索引 0；编号与 -lgip 一致）：

```
# 示例 A：创建一个 7g.40gb（整卡型单 GI，profile ID=0）
sudo nvidia-smi mig -i 0 -cgi 0 -C

# 示例 B：创建一个 1g.5gb（小切片，profile ID=19；可重复执行至 Free 为 0）
sudo nvidia-smi mig -i 0 -cgi 19 -C
```

创建后可用 **sudo nvidia-smi -L**、**sudo nvidia-smi mig -lgi** 查看；**Compute Instance** 用 **nvidia-smi mig -cci ...**（见 **nvidia-smi mig -h**）。**K8s + GPU Operator** 场景下，具体 profile 与 **device-plugin 暴露的 MIG 设备名** 以 NVIDIA 文档与 Operator 版本为准。

8.3 GPU Operator（Helm）

NGC Helm 仓库在国内可能不稳定，失败时同样使用 **代理** 或在内网同步 chart：

```
helm repo add nvidia https://helm.ngc.nvidia.com/nvidia
helm repo update

helm upgrade --install gpu-operator nvidia/gpu-operator \
  --namespace gpu-operator \
  --create-namespace
```

MIG 相关 --set 以 NVIDIA 官方文档与 Chart values.yaml 为准。

控制面节点参与 GPU / MIG（Master 也调度 GPU 组件）： Chart 里 **daemonsets.tolerations** 会写入 **ClusterPolicy**，并作用于 **toolkit**、**device-plugin**、**MIG Manager**、**DCGM Exporter**、**GFD** 等 Operand DaemonSet。默认值通常**只**容忍 **nvidia.com/gpu** 的 **NoSchedule**，**不包含 node-role.kubernetes.io/control-plane**（或旧版 **master**）污点。若 GPU 在 **Master / 控制面** 上，未追加该容忍时，这些 Pod **不会**落到该节点——常见现象是：节点已打 **nvidia.com/mig.config**，但 **nvidia.com/mig.config.state 长期为空**；**kubectl get pods -n gpu-operator -o wide | grep <控制面节点名>** 看不到 **nvidia-mig-manager**。处理方式：在 **helm upgrade** 中保留原有 **nvidia.com/gpu** 容忍，并**追加**控制面容忍，例如使用 **values 文件**（推荐）：

```yaml
# gpu-operator-values.yaml 片段（与 Chart 默认对齐后再追加）
daemonsets:
  tolerations:
    - key: nvidia.com/gpu
      operator: Exists
      effect: NoSchedule
    - key: node-role.kubernetes.io/control-plane
      operator: Equal
      value: ""
      effect: NoSchedule
    # 若集群仍使用旧污点名，可保留下一行
    - key: node-role.kubernetes.io/master
      operator: Equal
      value: ""
      effect: NoSchedule
```

```
helm upgrade --install gpu-operator nvidia/gpu-operator \
  --namespace gpu-operator \
  -f gpu-operator-values.yaml
```

升级后确认 **nvidia-mig-manager**、**nvidia-device-plugin-daemonset** 等在控制面节点为 **Running**；再按 **§8.6** 检查 **mig.config.state**。控制面上的 **containerd NVIDIA runtime**、**nvidia-persistenced** 等仍须按 **§8.1 / §8.5** 在**该节点**本机满足，与 Worker 相同。

控制面（Master）与 Worker 的 MIG「管理」对齐： GPU Operator 对 MIG 的**集群级**设置（**ClusterPolicy**：如 **mig.strategy**、**daemonsets.tolerations**）不区分 Master/Worker，**一份 Helm values 即全局生效**。若希望 **Master** 上「开 **MIG**、与 **Worker 同一策略**」，还需两点与 Worker **一致**：（1）**每节点标签 nvidia.com/mig.config=<键名>** 在 **Master** 上与 **Worker** 使用**同一取值**（例如均为 **all-balanced** 或均为 **all-disabled**；键名须落在 **各自 节点名-mig-config** 的 **mig-configs** 内，见 **§8.6**）；（2）**宿主机上 MIG 模式 / 驱动 / toolkit** 与 Worker 同级就绪（**§8.1 / §8.5**）。仅 Master 额外需要 **daemonsets.tolerations** 含 **control-plane**，否则 **MIG Manager / device-plugin** 不会落在控制面，**mig.config.state 也不会更新**。

```
kubectl get pods -n gpu-operator
kubectl describe node <gpu 节点名> | grep -i nvidia
```

8.4 GPU Operator 涉及哪些镜像（ErrImagePull 时）

以你集群为准（必跑）： 列出 gpu-operator 命名空间里 **Pod 实际引用的 image:tag**（含 initContainer）：

```
kubectl get pods -n gpu-operator -o yaml | grep " image:" | sed 's/.*image: //' | sort -u
```

或（需节点已装 jq）：

```
kubectl get pods -n gpu-operator -o json | jq -r '.. | objects | select(has("image")) | .image' | sort -u
```

单 Pod 失败原因：

```
kubectl describe pod -n gpu-operator <pod 名>
```

默认来源（Chart 默认值会随版本变化；标签以 helm show values / 上表为准）：

组件	常见镜像仓库	说明
Node Feature Discovery（你当前多为此报错）	**registry.k8s.io/nfd/node-feature-discovery**	大陆环境常 **ImagePullBackOff**，需**镜像加速 / 同步到私有仓库** 后改 node-feature-discovery.image
GPU Operator 控制器	**nvcr.io/nvidia/gpu-operator**	NGC 公共仓库
Validator	**nvcr.io/nvidia/gpu-operator**	同上
Driver / Toolkit / device-plugin / DCGM Exporter / MIG Manager 等	**nvcr.io/nvidia/...**、**nvcr.io/nvidia/k8s/...**、**nvcr.io/nvidia/cloud-native/...**	见当前 Chart **helm show values nvidia/gpu-operator**

查看当前 Chart 默认版本与镜像键：

```
helm show values nvidia/gpu-operator | grep -E 'repository:|image:|version:|tag'
```

大陆网络： 对 **registry.k8s.io** 的 NFD 可仿照 **§6.4**：在可访问环境 **crictl pull / ctr pull** 后推到 **华为 SWR** 等，再在 **helm upgrade** 里设置 **node-feature-discovery.image.repository / tag**（**tag 须与官方一致**）。**nvcr.io** 拉不动时同理同步 **nvcr.io/nvidia/...**，或通过 **代理** 让节点拉取。

8.5 no runtime for "nvidia" is configured（GPU Operator Pod 卡在 Sandbox）
现象： kubectl describe pod 的 **Events** 出现 **Failed to create pod sandbox: ... no runtime for "nvidia" is configured**；Pod 带 **Runtime Class Name: nvidia**。说明 **该节点上的 containerd 未配置 NVIDIA 容器运行时**，与镜像是否拉取成功无关。

处理（在报错节点上执行，例如 Worker ubuntu-6097）：

1. 确认已安装 **NVIDIA Container Toolkit**（见 **§8.1**），且 **nvidia-container-toolkit 的 DaemonSet** 已在该节点完成挂载/配置（可先 **kubectl get pods -n gpu-operator -l app=nvidia-container-toolkit-daemonset -o wide** 看对应节点是否为 **Running**）。
2. 在节点上 **写入 NVIDIA runtime 并重启 containerd**（路径按本机为准，常见为 **/etc/containerd/config.toml**）：

```
sudo nvidia-ctk runtime configure --runtime=containerd
sudo systemctl restart containerd
sudo systemctl restart kubelet
```

若 **nvidia-ctk** 报 **unrecognized runtime 'containerd'**，见 **§8.1** 末尾：**升级到 toolkit 1.14+** 或 **按 NVIDIA 文档手动编辑 config.toml** 后再 **systemctl restart containerd**。

3. 若仍报错，检查 **/etc/containerd/config.toml** 中是否存在 **[plugins."io.containerd.grpc.v1.cri".containerd.runtimes.nvidia]**（或 toolkit 生成的等效 **nvidia** runtime 段）；**勿**与旧版 **runc** 配置冲突。合并配置后再次 **systemctl restart containerd**。
4. 集群侧确认存在 **RuntimeClass**：

```
kubectl get runtimeclass nvidia -o yaml
```

注意： **每个 GPU Worker** 都需具备上述 containerd + toolkit 能力；若只在 Master 执行过 **§8.1** 的 **nvidia-ctk**，**Worker 必须单独做一遍**（或由 **nvidia-container-toolkit-daemonset** 成功落盘配置后再 **restart containerd**）。

/run/nvidia-persistenced/socket: no such file or directory（dcgm-exporter** / **device-plugin** 等 **RunContainerError**）：** 节点上 **nvidia-persistenced** 未运行，容器挂载 socket 失败。执行 **sudo systemctl enable --now nvidia-persistenced**，**ls -la /run/nvidia-persistenced/** 应存在 **socket**；再删掉该节点 **gpu-operator** 里异常 Pod 以重建。若曾为 MIG 调试 **stop nvidia-persistenced**（见 **§8.2**），结束后务必 **start/enable**，否则 NVIDIA 组件易起不来。

8.6 从 nvidia.com/mig.config=all-disabled 改为启用 MIG 切片（GPU Operator）
含义： **all-disabled** 表示由 **MIG Manager** 管理的几何为「**不按 MIG 切片对外暴露**」（与节点 BIOS/驱动里是否打开 **MIG 模式** 是不同层面；调度 **MIG 设备** 需改该标签）。详见 GPU Operator — MIG。

步骤概要：

1. **查看该节点可用的配置名**（GPU Operator **v26.x** 会为 MIG 节点生成名为 **节点名-mig-config** 的 ConfigMap，内含 **all-disabled**、**all-balanced** 及 **all-<profile>** 等键名）：

```
kubectl get configmap -n gpu-operator | grep mig-config
kubectl get configmap -n gpu-operator ubuntu-6097-mig-config -o yaml   # 将 ubuntu-6097 换成你的节点名
```

在 **data.config.yaml**（或 **config.yaml**）的 **mig-configs:** 下**已有**的键名才是合法取值（例如 **all-1g.5gb**、**all-7g.40gb**、**all-balanced**，**以你集群 ConfigMap 为准**）。

2. **给节点打标签**（示例：整卡多切片常见 **all-1g.5gb**；请换成上一步里**确实存在**的键）：

```
kubectl label node <节点名> nvidia.com/mig.config=all-1g.5gb --overwrite
```

3. **观察状态**：MIG Manager 会**暂时停掉**相关 GPU Pod，重配后再拉起。直到标签变为成功：

```
kubectl get node <节点名> -o jsonpath='{.metadata.labels.nvidia\.com/mig\.config\.state}{"\n"}'
# 期望最终为：success
```

4. **确认可分配资源** 已从 **nvidia.com/gpu: 1** 变为 **多个 nvidia.com/mig-...**（名称与 profile 一致），再按 **ClusterPolicy 里的 mig.strategy**（**single** 或 **mixed**）编写 **resources.limits**。**single** 要求**该节点上**所有 MIG 设备为**同一** profile；**mixed** 允许同节点多种 profile（见下条故障）。

改回关闭 MIG 调度几何：

```
kubectl label node <节点名> nvidia.com/mig.config=all-disabled --overwrite
```

注意： 切换配置可能导致**短时无 GPU Pod**；生产环境先 **cordon / 腾空**再操作。自定义几何需 **ConfigMap + ClusterPolicy** 等，以 NVIDIA 文档 **Custom MIG** 小节为准。

MIG 应用失败（nvidia.com/mig.config.state=failed，mig-manager 日志含 In use by another client / could not be reset）： 切换 **MIG 模式** 或 **reset GPU** 时要求 **GPU 无任何占用**。在 **对应 GPU 节点**上：**sudo nvidia-smi** 看 **Processes**；停桌面 **sudo systemctl isolate multi-user.target** 或 **stop gdm3**；可 **sudo systemctl stop nvidia-persistenced** 后再试；多卡 **SXM** 环境注意 **Fabric Manager**。确保 **无业务/监控进程** 占 GPU 后，可 **kubectl delete pod -n gpu-operator --field-selector spec.nodeName=<节点名>** 清掉本节点 gpu-operator Pod，再 **重新打 nvidia.com/mig.config**（或先 **all-disabled** 再切回 **all-balanced**）。仍失败时 **sudo reboot** 后趁**未起桌面**再应用配置。

device-plugin 日志 more than one MIG device type present on node（CrashLoopBackOff，**MIG_STRATEGY=single**）： 表示 **device-plugin** 在 **mig.strategy: single**（Chart 默认）下发现该节点存在**多种** MIG profile（例如 **all-balanced** 或手工 **nvidia-smi mig -cgi** 混用了不同切片）。**处理：** 在 **gpu-operator-values.yaml** 顶层增加 **mig.strategy: mixed**（与 **daemonsets** 等并列），例如：

```
mig:
  strategy: mixed
```

再执行 **helm upgrade gpu-operator ... -f gpu-operator-values.yaml**（**ClusterPolicy** 会同步为 **mixed**，通常整集群一项即可）。**或** 在该节点把几何改为**仅一种** profile（例如 **all-disabled** 后只选 **all-7g.40gb** 等单一 **all-<profile>**），再保持 **single**。

9. 验证命令汇总

```
kubectl get nodes -o wide
kubectl get pods -n kube-system
kubectl get pods -n monitoring
kubectl get storageclass
kubectl cluster-info
```

10. 防火墙（按需，UBUNTU UFW 示例）

Master：

```
sudo ufw allow 6443/tcp
sudo ufw allow 10250/tcp
sudo ufw allow 2379:2380/tcp
sudo ufw allow 10259/tcp
sudo ufw allow 10257/tcp
```

所有节点：

```
sudo ufw allow 10250/tcp
```

NFS 服务器：

```
sudo ufw allow from 192.168.10.0/24 to any port nfs
sudo ufw allow from 192.168.11.0/24 to any port nfs

sudo ufw enable
sudo ufw status
```

11. 参考链接

- Kubernetes kubeadm：https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/install-kubeadm/
- 清华 Kubernetes 镜像说明：https://mirrors.tuna.tsinghua.edu.cn/help/kubernetes/（将文档中的 v1.28 换为 **v1.30**）
- Calico：https://docs.tigera.io/calico/
- nfs-subdir-external-provisioner：https://github.com/kubernetes-sigs/nfs-subdir-external-provisioner
- kube-prometheus-stack：https://github.com/prometheus-community/helm-charts
- NVIDIA GPU Operator：https://docs.nvidia.com/datacenter/cloud-native/gpu-operator/
- NVIDIA Container Toolkit（apt / signed-by）：https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/install-guide.html